Access-Induced Semantic Divergence in Generated Program Transformations

Abstract

Access-induced semantic divergence occurs when an operation that appears observational updates latent state and changes a later externally visible result. We study this phenomenon through an observation-sensitive deterministic-state (OSDS) model, proof obligations for a straight-line core, real-package witnesses, and coding-agent transformations. The real-code study contains 20 confirmed divergences across 12 unmodified PyPI packages and 9 caller-level branch flips. A frozen coding-agent benchmark built from those witnesses evaluates behavior preservation under ordinary tests and OSDS-aware metamorphic checks. In the primary benchmark, gpt-5.6-sol produced 0 verified OSDS divergences in 26 tasks, gpt-5.6-terra produced 2, and gpt-5.6-luna produced 3. All five verified failures passed ordinary tests. A causal-control replay reproduced all five failures and removed all five when the identified access-induced mechanism was neutralized. A prospectively frozen 7-witness expansion adds 14 tasks and 42 Sol/Terra/Luna generations; all 42 preserved behavior. The results support a bounded claim: generated transformations can silently violate access-sensitive semantics, and OSDS-aware checks plus mechanism controls make such failures reproducible and falsifiable.

1. Introduction

Many program transformations treat reads, logging, inspection, representation, and caching as harmless local changes. That assumption fails when the accessed object carries latent state. A read may advance a cursor, update a cache statistic, materialize a stream, change a handler level, or populate an identity map. The later program can then observe a different value even when the added operation did not appear to change the explicit return value at the point where it was inserted.

This paper studies this failure mode as access-induced semantic divergence. The term does not mean that every observation is unsafe. It identifies a semantic pattern: an operation with an observational surface changes latent state that a later read can expose. The target setting is software verification for generated transformations, where ordinary tests can pass while an access-sensitive metamorphic check fails.

The contribution is a combined formal and empirical account. We give an OSDS model for read and observation transitions, state proof obligations for deterministic and zero-divergence cases, reproduce real-package divergences, and evaluate coding-agent transformations against frozen tasks derived from those witnesses. The coding-agent study is not a prevalence estimate. It is a controlled test of whether generated behavior-preserving edits respect access-sensitive semantics.

The verification setting matters because the requested edits are usually phrased in ordinary software-engineering terms: add instrumentation, cache a representation, simplify repeated access, move a helper, or expose a diagnostic value. These instructions sound behavior-preserving when the accessed API is treated as observational. OSDS cases violate that premise through state that is outside the surface syntax of the edit. The failure can therefore survive ordinary unit tests that exercise one order of execution and miss the counterfactual order in which observation and read commute differently.

The empirical strategy has three layers. First, a static screen and manual review identify candidate access-sensitive APIs in real packages. Second, package-shaped metamorphic harnesses confirm which candidates actually diverge under exact versions and caller wrappers. Third, frozen coding-agent tasks ask for natural transformations around those witnesses and replay the generated code without interpretation. This layering is intentionally conservative. Static screening supplies a review queue, the real-code oracle supplies executable witnesses, and coding-agent replay supplies behavioral evidence for generated transformations.

The paper makes three bounded claims. Access-induced divergence is a real semantic pattern in deployed packages. Coding agents can produce transformations that silently trigger that pattern. Mechanism-neutralizing controls can

separate these semantic failures from ordinary programming mistakes. The prospective expansion found no new failures, which constrains the breadth of the coding-agent claim and clarifies that the main contribution is mechanism-grounded verification evidence rather than a broad rate estimate.

2. OSDS Model

An OSDS value has a stable component and latent access state. In the proof core a semantic value is (b, a, d), where b is stable, a records read count, and d records latent drift. A read transition exposes f(b, a, d) and updates latent state. An observation transition exposes no additive value but updates latent drift through a deterministic function g. A program body folds those transitions over an operation list and applies a deterministic cap.

This model separates exposed values from latent effects. A logging call, representation call, cache lookup, or stream inspection can be modeled as an observation when it contributes no intended body value but may update d. Divergence appears when two orderings contain the same operations but reach a later read with different latent state.

We use two witness roles. Hidden-observation witnesses are cases where the access looks locally observational in the generated edit and the state-dependent effect appears later. Expected-access-sensitive witnesses are calibration cases where a careful programmer should notice that the operation consumes or mutates access state, such as stream reads or cursor movement. The distinction is important for interpretation. A failure on a hidden-observation row is evidence that the model treated the observation as harmless. A failure on an expected-access-sensitive row may still be a serious bug, but it does not by itself demonstrate the hidden OSDS pattern.

The metamorphic oracle compares two executions that differ in the placement of the observation-shaped access. The baseline and transformed program are evaluated on package objects with matched initialization. Ordinary smoke tests check direct user-facing behavior in a single execution order. OSDS-aware checks ask whether the transformation preserves behavior across the access order that the formal model marks as relevant.

At the level of a transition diagram, the pattern is:

Step	Original path	Transformed path	Divergence condition
initialize	construct object with latent state	construct matched object	states equivalent
observe	no extra observation	perform observation-shaped access	transformed latent state may drift
read	later user-visible read	later user-visible read	exposed values differ
classify	ordinary output may pass	OSDS oracle compares both paths	verified after manual review

The proof appendix establishes four bounded facts for the studied straight-line template: fixed-order determinism, zero divergence for identity observations, zero divergence for access-insensitive reads, and preservation of body-level divergence under a nonzero-slope linear cap. The proofs deliberately avoid claims about arbitrary Python programs, analyzer soundness, or production prevalence.

Two additional structural results guide the empirical design. The linear-cap preservation lemma gives a sufficient condition under which a body-level difference survives the final cap, so oracle checks need to include the post-cap visible result. The adjacent-swap result shows that for the restricted core, the extrema of access-order effects can be witnessed by neighboring order changes; this justifies focusing harnesses on compact before/after observation placements. A restricted two-dimensional range-degree result bounds the search in the finite template used by the formal experiments. These are proof obligations for the abstraction, not claims that Python control flow is globally enumerable.

3. Real-Code Evidence

The real-code oracle study instantiates the OSDS transition structure on unmodified package operations. From 60 selected candidates, 39 executable harnesses were constructed. The metamorphic oracle found 20 confirmed divergences across 12 packages: httpcore, PyYAML, pytest, markdown, more-itertools, docutils, beautifulsoup4, boltons,

cerberus, dnspython, h11, and anyio.

The divergences include stream materialization in httpcore.Response, identity-cache reuse in PyYAML, handler-level mutation in pytest, reference-registry mutation in markdown, cursor advance in more-itertools and dnspython, destructive tree extraction in beautifulsoup4, cache recency and statistics in boltons, validation-error population in cerberus, and buffer consumption in h11.

Nine confirmed divergences were lifted into caller-level branch flips. Each wrapper changed both a branch label and a downstream consequence, such as cache versus stream handling, alert emission versus suppression, request acceptance versus rejection, and recomputation versus cache serving. Nineteen negative controls removed the divergence under fresh-object, reset-between, or pure-observation interventions.

The discovery pipeline began with 73 PyPI packages and 278 reviewed MEDIUM/HIGH static findings. Manual review labeled 203 likely true positives and 75 likely false positives, for reviewed precision 0.7302. A rebuilt exact-version source snapshot contained 4,383 analyzable classes. A deterministic sample of 200 unflagged classes in that rebuilt snapshot found 0 likely missed matches and 200 likely nonmatches. These static numbers are reported as screening and audit evidence. They are not used as a prevalence estimate because the reviewed set is enriched by the detector and the rebuilt snapshot is not byte-identical to the original package corpus.

Executable confirmation is the decisive real-code filter. The oracle selected 60 candidates from the reviewed pool, attempted harness construction for all 60, and retained 39 executable package-shaped harnesses. Twenty of those harnesses diverged under the metamorphic relation. Nineteen constructed harnesses did not diverge, and 21 candidates failed construction, import, relevance, or safety checks. This denominator keeps the real-code result falsifiable: a static finding matters for the paper only when a concrete package version and harness can be replayed.

Real-code item	Count
Reviewed static findings	278
Packages in static screen	73
Rebuilt analyzable classes	4,383
Sampled unflagged classes	200
Candidate harnesses attempted	60
Executable harnesses	39
Confirmed divergences	20
Confirmed packages	12
Caller-level branch flips	9
Divergence-removing controls	19/19

The caller-level wrappers answer a verification question that raw value differences alone do not settle. When an access changes a later value and callers ignore that value, the software consequence may be narrow. The wrappers show that nine confirmed divergences can alter caller decisions. The result is a transformation-preservation claim rather than a defect claim about the original packages: transformations which relocate or duplicate accesses must preserve latent-state behavior when callers depend on it.

4. Coding-Agent Benchmark

The primary coding-agent benchmark was frozen before model execution. It contains 13 base tasks and 26 normal/warned prompt variants from 9 packages and 9 unique witnesses. Each task asks for a small behavior-preserving Python transformation. The model sees only the code and the editing instruction. It does not see oracle labels, witness labels, benchmark explanations, or prior responses.

Evaluation uses exact-response replay. The runner extracts Python, executes ordinary smoke tests, then compares the baseline and candidate under an OSDS-aware metamorphic oracle. Failures are manually classified as verified semantic

divergence, ordinary programming bug, invalid patch, environment failure, oracle issue, or unclear. A verified OSDS divergence requires an executable transformation, real behavioral divergence, OSDS oracle detection, and manual confirmation that the mechanism is access-induced.

The 26 prompt variants cover six transformation families: instrumentation, caching/materialization, refactoring, repeated-access cleanup, access reordering, and debugging/inspection. Each base task has a normal prompt and a warned prompt. The warned prompt states that repeated access or observation may affect behavior, but it does not expose the oracle output or witness label. All generation tasks were fresh and projectless. Self-assessment tasks, where available in the primary study, were also fresh and blinded to the oracle.

The replay pipeline treats generated text as data. Raw responses are saved exactly, then code extraction, patch application, ordinary tests, OSDS checks, and manual classification are run from JSONL artifacts. This avoids interactive repair. It also separates invalid patches from semantic divergences: a syntactically broken edit can be a model failure, but it is not counted as OSDS evidence.

Evaluation stage	Purpose	Counted failure classes
raw response capture	preserve exact model output	none
extraction and patching	obtain executable candidate	invalid patch
ordinary smoke tests	check direct task behavior	ordinary programming bug
OSDS-aware replay	compare access-order semantics	candidate semantic divergence
manual review	attribute mechanism	verified OSDS, oracle issue, unclear

This design makes ordinary tests a measured baseline rather than a straw target. The ordinary tests encode the direct behavior requested in the prompt. They are useful for separating unrelated programming defects from preservation errors. Their limitation is that they do not quantify over the observation order that OSDS marks as semantically relevant.

5. Primary Model Results

The primary benchmark evaluated three Codex task-model configurations at low reasoning effort with temperature and seed unavailable through the task interface. gpt-5.6-sol produced 26 executable candidates, with 23 behavior-preserving candidates, 3 ordinary programming bugs, and 0 verified OSDS divergences. gpt-5.6-terra produced 24 executable candidates, with 18 behavior-preserving candidates, 4 ordinary bugs, 2 invalid patches, and 2 verified OSDS divergences. gpt-5.6-luna produced 24 executable candidates, with 17 behavior-preserving candidates, 4 ordinary bugs, 2 invalid patches, and 3 verified OSDS divergences.

Model	Executable	Preserved	Ordinary bugs	Invalid patches	Verified OSDS	Silent OSDS
gpt-5.6-sol	26	23	3	0	0	0
gpt-5.6-terra	24	18	4	2	2	2
gpt-5.6-luna	24	17	4	2	3	3

All five verified OSDS failures were silent under ordinary tests. Terra failed on pytest_catching_logs__instrumentation__normal and pytest_catching_logs__instrumentation__warned. Luna failed on those two pytest rows and on pyyaml_representer__caching_materialization__normal. Each failure appeared in a hidden-observation case. The expected-access-sensitive calibration rows produced ordinary bugs or invalid patches, with no verified OSDS divergence.

The verified failures are five model-task outcomes over two underlying package mechanisms. The pytest failures occur in both normal and warned instrumentation prompts for Terra and Luna. The PyYAML failure occurs for Luna on the normal caching/materialization prompt. This distinction matters for evidence accounting: the benchmark contains five verified generated failures, while the underlying package spread for those verified OSDS failures is two packages and two witness families.

Evidence role	Model rows	Preserved	Ordinary bugs	Invalid patches	Verified OSDS
hidden observation	48	43	0	0	5
expected access-sensitive	30	15	11	4	0

The role split supports the main interpretation. The hidden-observation rows contain all verified OSDS failures and no ordinary bugs in the completed primary manual review. The expected-access-sensitive rows contain ordinary bugs and invalid patches, which are useful engineering outcomes but not evidence of the hidden-observation phenomenon. This is why the paper reports behavioral preservation, ordinary defects, invalidity, and OSDS divergence separately.

Self-assessment was conservative in the primary study. Across Sol, Terra, and Luna, there were zero false YES preservation claims on verified OSDS divergences. The self-assessment results are secondary because the primary evidence is behavioral replay.

The primary self-assessment counts were Sol 7 YES and 19 NO, Terra 6 YES and 20 NO, and Luna 8 YES and 18 NO. Conservative NO responses were common: Sol had 16, Terra had 12, and Luna had 9. These results should not be read as calibration of model introspection in general because the assessment prompt was narrow and blinded. They do show that the five verified OSDS failures were not accompanied by false preservation claims in this study.

6. Causal Controls

The causal-control experiment replays the exact generated candidate files for the five verified failures. The candidate source remains byte-identical. The intervention changes only the witness or environment mechanism that caused the access-induced divergence.

For pytest, the control isolates diagnostic logging from the captured handler hierarchy so that the logging-shaped observation no longer mutates the handler state read later by the program. For PyYAML, the control clears the relevant identity/access cache state after the representer observation while retaining the generated caching transformation. Under the original witness environment, all five failures reproduce: ordinary tests pass and OSDS checks fail. Under the mechanism-neutralizing control, all five OSDS divergences disappear. The causal status for all five rows is mechanism_neutralized_divergence_disappeared.

This result is important because it narrows the explanation. The failures are mechanism-dependent generated-code defects. They are reproduced by the exact generated transformations and removed by targeted neutralization of the access-induced mechanism.

Failure family	Rows	Original replay	Controlled replay
pytest catching_logs instrumentation	4	ordinary pass, OSDS fail	OSDS pass
PyYAML representer caching	1	ordinary pass, OSDS fail	OSDS pass

The control criterion is asymmetric. A control that merely changes the task until both baseline and candidate pass would be weak evidence. Here the generated candidate is unchanged, the original witness replay still fails, and the mechanism-neutralized replay passes. That pattern supports the causal statement that the access-induced mechanism is necessary for the observed divergence in these five rows.

7. Prospective Expansion

After the causal-control phase, the unused confirmed real-code witness pool was audited prospectively. Eleven confirmed witnesses were unused by the primary benchmark. Seven met the eligibility criteria: exact package reconstruction, baseline execution, oracle reproduction, caller/control availability where applicable, and automated task execution. The expansion was frozen at commit 0f7dea5ca3cfc62e040026a8c780f1127526b0dc before any model execution.

The frozen expansion contains 7 new base tasks and 14 normal/warned variants from 3 packages and 7 witnesses: boltons LRI statistics, boltons MultiFileReader, h11 ReceiveBuffer, boltons SpooledStringIO, boltons SpooledBytesIO, dnspython Tokenizer concatenation, and a second boltons LRU pair witness. All expansion rows are expected-access-sensitive calibration cases.

Sol, Terra, and Luna were run on the exact frozen expansion prompts as fresh projectless Codex tasks. Self-assessment was skipped in the cut-scope completion. The three replays produced 42 executable candidates. All 42 passed ordinary tests and OSDS-aware checks. No expansion row was classified as an ordinary bug, invalid patch, verified OSDS divergence, silent divergence, environment failure, oracle issue, or unclear. Normal prompts preserved behavior in 21/21 rows; warned prompts preserved behavior in 21/21 rows.

Model	Tasks	Executable	Preserved	Verified OSDS
gpt-5.6-sol	14	14	14	0
gpt-5.6-terra	14	14	14	0
gpt-5.6-luna	14	14	14	0

The expansion includes 3 packages and 7 witnesses, with package spread concentrated in boltons, dnspython, and h11. It was designed as a prospective check against overfitting the primary results. The outcome is negative for new OSDS failures: no additional verified divergence appeared. This is a meaningful result because the prompts and evaluation were frozen before execution, and the raw model responses were replayed through the same unchanged JSONL pipeline.

The exclusions are also informative. Four unused confirmed witnesses were excluded because exact reconstruction, stable harness execution, or automation constraints were not satisfied under the cut-scope completion criteria. Where applicable, they remain real-code evidence in the earlier oracle study. The prospective coding-agent count includes only the seven eligible frozen witnesses.

8. Model Differential Findings

The primary benchmark contains the model-differential evidence. Sol preserved all verified-OSDS rows where Terra or Luna diverged. Terra diverged on two pytest instrumentation rows. Luna diverged on those same two rows and on one PyYAML caching row. The prospective expansion contains no model-differential OSDS failures: all 14 tasks were preserved by Sol, Terra, and Luna.

The expansion result should be read as a boundary, not a contradiction. The added witnesses were expected-access-sensitive cases where the access-sensitive operation is visible in the code shape. The primary failures occurred in hidden-observation cases where a generated logging or caching edit looked locally harmless and ordinary tests did not expose the later state-dependent effect.

The differential pattern is small but useful. It identifies transformations where one Codex task-model configuration preserved the OSDS relation and another did not under the same frozen prompt. In pytest instrumentation, Sol preserved both variants while Terra and Luna inserted logging-shaped accesses that changed the later handler-level behavior. In PyYAML caching/materialization, Sol and Terra preserved the normal variant while Luna introduced reuse across representer identity state. The result motivates model-comparative verification because a single pass/fail aggregate can hide which witness families are configuration-sensitive.

9. Verification Implications

Ordinary smoke tests are insufficient for this class of transformations. The five primary OSDS failures passed ordinary tests because the visible single-order behavior remained plausible. The OSDS oracle detected divergence by comparing orderings that differ only in the placement of an observation-shaped access on an equivalent object.

The causal controls provide a practical validation pattern. A semantic-preservation claim is stronger when a failure reproduces under the original witness and disappears under a mechanism-neutralizing intervention that leaves the generated candidate unchanged. This pattern helps distinguish access-induced semantic divergence from unrelated programming bugs.

For verifiable coding, the direct lesson is that preservation checks need to model access as part of semantics. A transformation that adds an apparently harmless inspection can be invalid when the inspected object carries hidden state. Verification systems for generated patches should therefore include metamorphic relations over observation placement, explicit object-state setup, and checks that compare both ordinary behavior and access-order behavior.

The result also suggests a triage workflow. Ordinary tests remain useful for filtering invalid patches and unrelated programming defects. OSDS-aware checks then target transformations involving logging, representation, serialization, stream reads, caches, iterators, handlers, and mutable registries. Mechanism-neutralizing controls should be used for high-value failures before making a causal claim. This workflow is more demanding than a smoke test suite, but it produces evidence that can be replayed and falsified.

The formal model supplies a vocabulary for that workflow. A verifier can ask whether a proposed edit inserts or reorders an observation transition, whether the later read is access-sensitive, and whether the final visible result preserves a nonzero body-level difference. These questions map directly to the hidden-observation and expected-access-sensitive roles used in the benchmark.

10. Limitations

The coding-agent benchmark is small and correlated by witness and package. Counts are benchmark outcomes rather than prevalence estimates. Codex task-model runs are real-model results collected through Codex projectless tasks rather than the OpenAI API. Temperature and seed were unavailable through the task interface. The prospective expansion adds seven witnesses, all expected-access-sensitive, with no new hidden-observation witnesses.

Manual review remains part of the verified OSDS classification. The formal core covers a straight-line deterministic template and does not prove analyzer soundness or general Python semantics. The real-code study uses selected candidates and constructed harnesses, so it supports existence, mechanism, and reproducibility rather than ecosystem prevalence.

The primary verified OSDS failures come from two underlying package mechanisms, pytest and PyYAML. The five-row count reflects model and prompt variants over those mechanisms. The prospective expansion added seven witnesses and found zero new failures. That negative result reduces confidence in any broad claim about current Codex task-model failure frequency. It leaves intact the existence, silence, and causal-control results for the verified failures.

The study is Python-focused. It does not cover compiled languages, concurrent programs, distributed systems, or UI event systems where observation and mutation may have different structure. The ordinary smoke tests are benchmark-specific and should not be treated as representative of industrial test suites. The Codex configurations are task-model configurations reported by the Codex runtime, not independent vendors or API model families.

Some evidence depends on exact package reconstruction. The real-code study uses unmodified package code, but package environments age and dependency resolution can change. The source package therefore includes frozen prompts, raw response locations, replay summaries, and control scripts so that readers can inspect the evidence path rather than relying on prose counts alone.

11. Conclusion

Access-induced semantic divergence is a concrete risk for generated behavior-preserving transformations. In real package code, observation-shaped operations can change later externally visible behavior through latent state. In the primary coding-agent benchmark, five verified silent OSDS failures were found across Terra and Luna, all missed by

ordinary tests. Exact candidate replay plus targeted causal controls reproduced those failures and removed all five when the identified mechanism was neutralized. The prospective expansion found no new failures across Sol, Terra, and Luna, which constrains the claim and strengthens the paper. The strongest evidence is a reproducible semantic failure mode with real-code witnesses, model-generated instances, and mechanism-level controls.